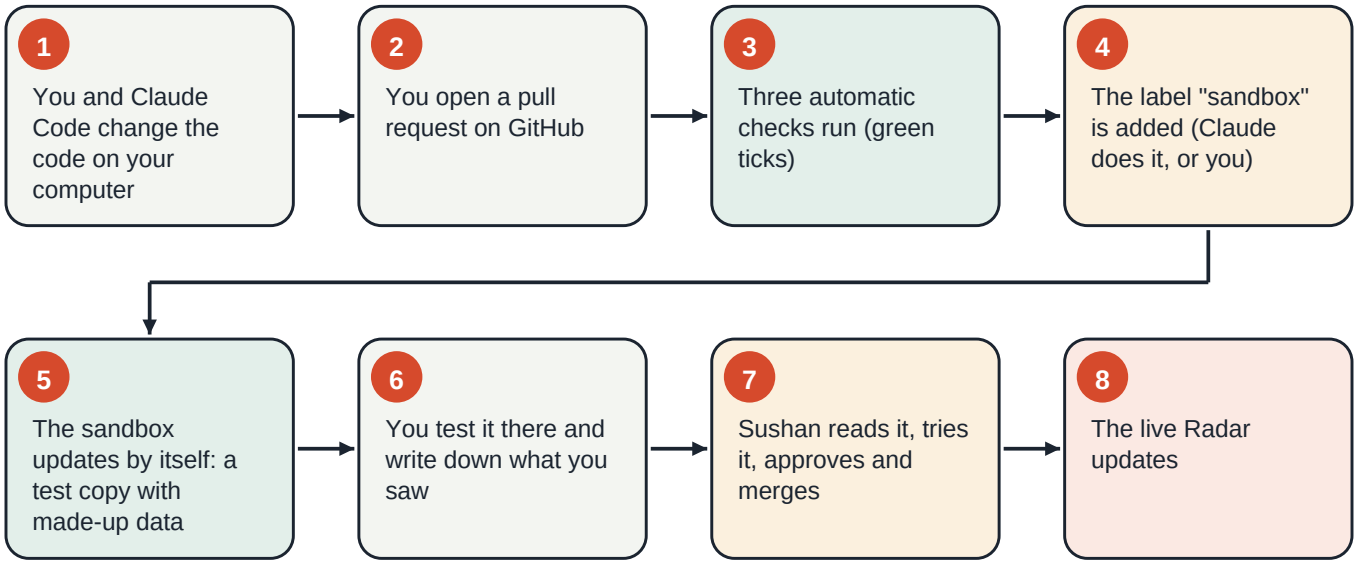
FOR EVERYONE WHO CHANGES THE CODE

Changing Radar with Claude Code

Setup, testing in the sandbox, and getting your change approved.

You write changes with Claude Code. Every change is tried on a safe copy of Radar (the sandbox) and read by Sushan before it goes live. Nobody can push straight to the live code, and that includes Sushan.

How a change travels



Good to know: The **live Radar** only changes when Sushan merges your pull request. The **sandbox** has made-up data, so nothing you do there can harm anything real. You can use the sandbox whenever you like, at any hour, so you never wait for Sushan to start testing. Only **Sushan** approves and merges, when he is available.

Who does what

You (the contributor)	Sushan (the approver)
Write the change with Claude Code.	Reads your pull request and the list of changed files, when he can.
Prove it builds and open the pull request.	Can free the sandbox (take the label off) when he needs to try something himself.
Let Claude add the sandbox label, test, and write down what you saw.	Tries it, then approves and merges.
Fix anything that was found.	Watches the live Radar after the merge, and can undo it.

Part 1: Setup, once

1. Send Sushan your GitHub username (make a free account at github.com if you have none). He invites you to **Rktlog/CRM**. Open the email and click **Accept invitation**.
2. Ask Sushan for the **sandbox logins** (one manager and three reps). They are made up, but keep them within the team.
3. Install **Git**, **Node.js 22** and **Claude Code** (setup is at code.claude.com/docs). You need your own Claude account that includes Claude Code.
4. Install the **GitHub CLI** (called **gh**, from cli.github.com) and sign in with your own account: type `gh auth login` and follow the questions. Claude Code uses it to open pull requests and to add the sandbox label for you.
5. Download the project: `git clone https://github.com/Rktlog/CRM.git` then `cd CRM`.
6. Install what the two apps need, once: `cd API`, `npm ci`, `cd ../Web`, `npm ci`, then `cd ..`.
7. Start Claude Code in the project folder by typing `claude`. It reads **CLAUDE.md** by itself. Check it did: ask *What rules do you follow in this project?* It should mention branches, pull requests, proof and the sandbox.
8. Do **not** ask for or use any live passwords or keys (Railway, Supabase, Vercel, DEAR, Gmail). You do not need them. The sandbox is where you test.

> **Good to know:** No Claude Code on your computer? You can use **claude.ai/code** in a browser instead: connect your GitHub account and pick Rktlog/CRM. If the repository does not appear in the list, use your own computer. The browser version may not be able to add labels: if Claude says so, add the label yourself (pull request page, right-hand column, Labels).

Part 2: Making a change, every time

1. Start fresh: **`git checkout main`** then **`git pull`**.
2. One branch for one job: **`git checkout -b yourname/short-topic`**.
3. Ask Claude Code for the change, using the prompt in the box below.
4. Read what it changed (**`git diff`**). You are responsible for what you send, not Claude.
5. Save and send it: **`git add -A`**, then **`git commit -m "what changed, in plain words"`**, then **`git push -u origin yourname/short-topic`**.
6. On GitHub click the yellow **Compare & pull request** bar and fill in the form. Under **Proof**, add a heading **How to try it in the sandbox** and paste the steps Claude gave you.
7. Wait for the three green ticks: **API build**, **Web build**, **No secrets**. If one is red, click Details, fix it and push again.
8. Go on to Part 3: put it in the sandbox and test it. When your test notes are on the pull request, tell Sushan it is **ready for review**. Do not merge it yourself.

Prompt to give Claude Code (fill in the first line):

```
Read CLAUDE.md. I want to change: [say it in plain words].
Make the smallest change that does it.
Then build both apps (API and Web) and show me the output.
When the three checks are green, put it in the sandbox (CLAUDE.md says how).
In your summary, list the exact steps I should click in the sandbox
to check it works, and what I should see at each step.
Do not touch anything under .github, docs or CLAUDE.md.
```

> **Tip:** Keep each change small. A small change is quick to read, quick to test in the sandbox, and quick for Sushan to approve.

Part 3: Testing in the sandbox

The sandbox is a separate copy of Radar with made-up shops, products and orders. Its address is **https://crm-snowy-ten-25.vercel.app**. After you sign in, a red **SANDBOX** bar shows at the top of every page. **If you ever see real shops and no red bar, you are on the live site: stop and tell Sushan.**

Login	What it sees
Manager	Every state, and the manager-only pages (Order Forecast & Procurement, Marketing).
Reps (three)	Only their own states. As set up: Margaret Wilson sees VIC, TAS, SA and WA; David Chen sees NSW and ACT; Priya Nair sees QLD. Ask Sushan which login is which.

Getting your change into the sandbox, and testing it

1. When the three ticks are green, **Claude Code adds the label sandbox for you** (CLAUDE.md tells it how, and it checks first that nobody else has the sandbox). If it cannot, add it yourself: your pull request page, right-hand column, **Labels**, tick **sandbox**. You never need to ask anyone. (If another pull request already has the sandbox, the label comes off again and a comment tells you who has it. See Rule 1.)
2. Within a minute a comment appears on your pull request: **The sandbox now holds this pull request at commit ...**. Check the code matches your latest commit. Then wait 3 to 5 minutes while Railway and Vercel rebuild.
3. Open the sandbox, press **Ctrl+F5**, and sign in with the login for the role your change affects.
4. Follow the **How to try it in the sandbox** steps from your pull request. Look at what you changed, then at the screens next to it.
5. Write down what you saw as a comment on your pull request (template below), with screenshots. Then tell Sushan it is ready for review.

What to check

No.	What to look for
1	It does what you said it should do.
2	The screens next to it still work (Dashboard, Customers, Orders, Products).
3	Odd cases: no results, a very long name, an empty list.
4	No red error messages on the page. (Optional: press F12, open Console. Red lines mean a problem.)
5	A screen change: also look at a narrow window, like a phone.
6	The right role: a rep must not see manager-only things.

Comment to paste on your pull request when you have tested:

```
TESTED IN THE SANDBOX
Login used: manager / rep (which one)
What I did: 1) ... 2) ... 3) ...
What I saw: ...
Result: works / does not work
Screenshots: attached
NOT tested here, and why: ...
```

Rule 1: One change at a time. The pull request that has the label holds the sandbox. If someone else has it, your label comes off again with a comment saying who. Wait for them to finish, or ask Sushan.

Rule 2: Pushing more changes updates the sandbox by itself (3 to 5 minutes), as long as your pull request still has the label.

Rule 3: Closing or merging your pull request frees the sandbox, and it goes back to the live code. Leave the label on until Sushan has tried your change, unless someone else is waiting. If you will be away for days, tell Sushan so he can free it.

Rule 4: Never type real customer details into the sandbox. It is for made-up data only.

What the sandbox cannot tell you

The sandbox has no real Cin7 (DEAR) sync, no real email, no Google sign-in or calendars, no AI assistant, no Claude connector, and a small amount of data. If your change depends on one of these, or on how it behaves with the real, much larger amount of data, say so under **Anything NOT proven** in your pull request. Sushan decides how to check it safely after the merge (a read-only look, never a guess).

Good to know: The sandbox database is on a free plan and **pauses after 7 days with nobody using it**. Nothing is lost, but the sandbox will not load data until someone clicks Restore project. If it cannot load, tell Sushan.

Good to know: If your pull request has a **merge conflict**, GitHub does not update the sandbox, and the label does nothing. Ask Claude: *Merge main into my branch, fix the conflicts and push.* Then add the label again.

Good to know: If the sandbox **crashes** with your change, you cannot see the server log. Tell Sushan, who can copy the Railway log to you. Never ask for the live logs.

Part 4: Getting approved

Sushan checks your pull request and tries your change in the sandbox himself. If he is happy he clicks **Approve** and then **Squash and merge**. The live Radar updates two to five minutes later. If he finds a problem he comments on the pull request: fix it and push. The sandbox updates by itself.

1. After it is merged, tidy your computer: `git checkout main`, `git pull`, then `git branch -D yourname/short-topic` (a capital D: git thinks a squashed branch is unmerged).
2. Start your next change from step 1 of Part 2.

The rules

Always	Never
Work on a branch and open a pull request.	Push to main (GitHub refuses) or merge your own pull request.
Prove the change: build output, sandbox test notes, screenshots.	Say it works if it was not run. Write "not proven" instead.
One job per pull request.	Mix a fix with a tidy-up.
Put changes to live data in a script file with a preview step, a backup and a row-count guard. Sushan runs it.	Run any script against the live database.
Use plain words in anything a rep or manager reads.	Commit passwords, keys or .env files, or anything you would not want strangers to read: this repository is public. The secrets check turns red for keys.
Leave .github, CLAUDE.md and docs alone unless the job is about them, and say so at the top of the pull request.	Push to the staging branch by hand, or change the checks so something passes.

Where to read more

File	What it is
CLAUDE.md	The house rules and the data rules: how dates work, what counts as a sale, costs are for managers only.
docs/data-notes/DATA_MAP.md	Read before touching demand, stock, on-order or sales numbers.
docs/SANDBOX.md	How the sandbox is built, and why it cannot touch live data.

> **Tip:** Stuck? Send Sushan three things: which step you were on, the red text or a screenshot, and the link to your pull request.

FOR THE APPROVER

Before you approve: your checklist

Use this for every pull request. It takes five to ten minutes for a small change. You do not need to be online while contributors test: review when you can.

1. Open the pull request. Read **What this changes**, **Why**, **Proof** and the contributor's **How to try it** steps. If you cannot follow them, send it back.
2. Click **Files changed**. Only the files you expect? The sandbox holds made-up data only, so anything is safe to try there. But read changes to these line by line before you approve, because they matter for live: **.github**, **CLAUDE.md**, **docs**, **API/prisma**, **API/scripts**, **API/src/mcp** and **API/src/lib/environmentGuard.ts**.
3. Check the three green ticks: **API build**, **Web build**, **No secrets**.
4. The contributor has probably put it in the sandbox already. Look for the comment from **github-actions** on the pull request, and check the **staging** branch shows their latest commit. If the sandbox holds a different pull request and you need it, take the label off that pull request (tell its owner), then add the label **sandbox** to the one you want to try. Nothing is lost: the other one adds the label again later. The rule is the same for you as for everyone, so nobody can take the sandbox by accident. Wait for Railway (staging) and the sandbox Vercel project to say **Ready**.
5. Try it yourself in the sandbox, following their steps, and one screen next to it. Read their test comment: does it match what you see?
6. If it changes data: the script or SQL must be a file in the pull request with a preview step, a backup and a row-count guard. **You** run it on the live database, never the contributor.
7. Click **Review changes**, **Approve**, **Submit review**. Then **Squash and merge** and **Delete branch** (never the staging branch).
8. Watch the live Radar for a few minutes after: the Railway deployment is healthy, the Dashboard loads, and the screen that changed looks right.
9. If something is wrong: open the merged pull request and click **Revert**. It makes a new pull request that undoes it. Check its ticks, then merge it the same way.